WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: ARTIFICIAL INTELLIGENCE

BACHELOR THESIS

SUPERVISOR:
Lect. Dr. Sebastian Ştefăniga

GRADUATE:
Raul Veca

TIMIŞOARA
2026

WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: COMPUTER SCIENCE IN ENGLISH

TrainingIT:

A Web-Based Application for Selling Programming Courses

SUPERVISOR:
Lect. Dr. Sebastian Ştefăniga

GRADUATE:
Raul Veca

TIMIŞOARA
2026

Contents

Abstract iii

Rezumat v

Statement of Scientific Integrity vii

1 Introduction 1
- 1.1 Objectives . . . 2
- 1.2 Structure of the thesis . . . 2

2 Problem Description 3
- 2.1 The customer side . . . 3
- 2.2 The business side . . . 3
- 2.3 The integration gap . . . 3
- 2.4 Problem statement . . . 4

3 Similar Approaches 5

4 Proposed Solution 7
- 4.1 One application, two experiences . . . 7
- 4.2 Everything is captured once . . . 7
- 4.3 Optional intelligence . . . 8

5 Application Architecture 9
- 5.1 Presentation layer (front end) . . . 9
- 5.2 Application layer (back end) . . . 10
- 5.3 Domain layer . . . 10
- 5.4 Persistence layer . . . 10
- 5.5 Role-based navigation . . . 11

6 Application Features 13
- 6.1 Account, authentication and roles . . . 13
- 6.2 The client portal . . . 13
  - 6.2.1 Courses (home page) . . . 13
  - 6.2.2 My courses . . . 13
  - 6.2.3 Online sessions and My sessions . . . 14
  - 6.2.4 Floating buttons: assistant and issue reporting . . . 14
- 6.3 The administrator portal . . . 14
- 6.4 Cross-cutting features . . . 14

6.5 Student Contributions 15

7 Technologies Used **17**
7.1 Front end 17
7.2 Back end 17
7.3 Persistence 17
7.4 Reporting and artificial intelligence 17

8 Implementation Details **19**
8.1 Design patterns in the domain layer 19
8.2 The event-driven core 19
8.3 Data model 20
8.4 AI integration 21
8.5 Real-time statistics and reporting 21

9 Testing **23**
9.1 Input validation 23
9.2 Robustness by design 23
9.3 Functional testing 23

10 Usage Examples **27**
10.1 A career changer, guided by the assistant 27
10.2 An advancing developer 27
10.3 A corporate client training a team 27
10.4 The purchase and review flow 28

11 Conclusions and Future Development Directions **29**
11.1 Conclusions 29
11.2 Future development directions 29

List of Figures **31**

A Appendix - Generative AI Tools Usage **35**
A.1 Claude Details 35

Abstract

TrainingIT is a web-based application that unifies two products that are usually kept separate: an online marketplace for selling programming and IT courses, and a Customer Relationship Management (CRM) system that records every customer interaction behind the scenes. Visitors browse a catalogue, buy courses in a single click, leave star reviews, and book one-to-one tutoring sessions; administrators manage contacts, sales, automatically generated invoices, analytics, partner-company employees, and support tickets. The client is built with Next.js and React; the server is a Spring Boot REST API whose domain layer is organised around classic object-oriented design patterns and persists to a MariaDB database. An optional integration with the Anthropic Claude API adds a course-recommendation assistant and live page translation. The thesis presents the problem, the proposed solution, the architecture, the technologies, the implementation, and the testing of the system.

Rezumat

TrainingIT este o aplicație web care reunește, într-un singur produs, două lucruri de obicei separate: un magazin online pentru vânzarea de cursuri de programare și IT și un sistem CRM (Customer Relationship Management) care înregistrează automat, în spate, fiecare interacțiune a clienților. Vizitatorii răsfoiesc un catalog, cumpără cursuri dintr-un singur clic, lasă recenzii cu stele și rezervă sesiuni de mentorat unu-la-unu; administratorii gestionează contacte, vânzări, facturi generate automat, rapoarte de analiză, angajații companiilor partenere și sesizările primite. Interfața este construită cu Next.js și React, iar serverul este un API REST realizat cu Spring Boot, al cărui nucleu de domeniu este organizat în jurul unor șabloane de proiectare clasice și persistă datele într-o bază de date MariaDB. O integrare opțională cu API-ul Anthropic Claude adaugă un asistent de recomandări și traducerea live a paginilor. Lucrarea prezintă problema, soluția propusă, arhitectura, tehnologiile, implementarea și testarea sistemului.

Statement of Scientific Integrity

I declare that this BSc thesis is an original report and represents my own work, conducted under the supervision of Lect. Dr. Sebastian Ștefăniga. All research methods, design decisions, implementation details, analysis, and interpretation presented in this thesis are accurately reported, and all external sources have been cited where I have made use of ideas, technical documentation, or visuals of others.

I used generative AI during the writing and research process. The details are provided in Appendix A.

Chapter 1

Introduction

The information-technology sector is one of the fastest-growing labour markets, and continuous learning has become a professional necessity rather than an option. As a result, the market for online programming courses has expanded rapidly, and training providers increasingly sell their courses directly on the web instead of relying only on classrooms. A training company that sells online, however, faces two problems at the same time. First, it needs an attractive storefront where visitors can discover courses, enrol, pay, and receive support. Second, it needs to understand and manage the people on the other side of that storefront: who they are, at what stage of the buying journey they are, what they purchased, how much revenue they generated, and what problems they reported. The first need is served by an e-commerce site; the second by a Customer Relationship Management (CRM) system. In practice these are usually two different pieces of software, connected-if at all-by fragile integrations.

This thesis presents **TrainingIT**, a web-based application that merges these two concerns into a single, coherent product. On the surface it is a modern marketplace for programming and IT courses: a visitor creates an account, browses a catalogue, buys a course with one click, leaves a review, and can book a private one-to-one tutoring session with a trainer. Underneath, every one of those actions is captured by a full CRM back office, where the company's team manages contacts and leads, monitors sales, issues invoices, studies analytics, onboards the employees of partner companies, and resolves support tickets. The same click that a customer makes on the storefront typically triggers several automated processes in the CRM-an invoice is produced, a lead score is recalculated, an audit entry is written-so that the heavy administrative work happens by itself.

The application is deliberately built as a demonstration of solid software engineering. The client is a Next.js and React single-page experience with a custom visual theme, a dark mode, and live translation into any language. The server is a Spring Boot REST API written in Java, whose domain layer is structured around classic object-oriented design patterns-Observer, Command, Strategy, Factory, Builder, Facade, and others-so that the system stays organised and predictable as it grows. All data is persisted in a MariaDB relational database. An optional integration with the Anthropic Claude language model adds a conversational course assistant, a recommendation quiz, per-company training recommendations, and live page translation; when no API key is configured, these features simply disappear and the rest of the application keeps working.

1.1 Objectives

The main objective of this work is to design and implement a production-shaped web application that sells programming courses while transparently maintaining a complete CRM record of customer activity. The concrete objectives are to provide a public client portal for browsing, purchasing, reviewing courses and booking one-to-one tutoring sessions; to provide an administrator portal that manages contacts, courses, purchases, invoices, analytics, corporate employees and support issues; to structure the server-side domain logic around well-known design patterns and an event-driven core, so that a single user action can propagate cleanly into several automatic reactions; to integrate an optional AI assistant for recommendations and live translation, degrading gracefully when it is disabled; and to persist all operational and CRM data in a relational database that survives restarts.

1.2 Structure of the thesis

Chapter 2 describes the problem the application addresses. Chapter 3 reviews similar existing approaches. Chapter 4 presents the proposed solution at a high level. Chapter 5 details the architecture, and Chapter 6 the user-facing features. Chapter 7 lists the technologies used, Chapter 8 the implementation details, and Chapter 9 the testing approach. Chapter 10 walks through concrete usage examples, and Chapter 11 draws conclusions and outlines future development directions.

Chapter 2

Problem Description

A company that sells IT training online has to satisfy two very different audiences with a single system, and it is the tension between them that defines the problem this thesis addresses.

2.1 The customer side

A prospective learner who arrives on the website expects the same smooth experience they get from any modern online shop. They want to see clearly what courses exist, how much they cost, what level they target, and what other learners thought of them. They want to create an account once, buy a course in a single click, and, if they need extra help, book a private session with a trainer at a time that suits them. If something does not work, they want a simple way to report it. Anything that adds friction-confusing navigation, broken availability calendars, reviews that cannot be trusted-costs the company a sale.

2.2 The business side

Behind the storefront, the training company has to run an operation. It must know who its potential customers (*leads*) are and how close each one is to buying, so that sales effort is spent where it matters. It must keep a reliable history of every purchase, produce correct invoices-including negotiated discounts for the employees of partner companies-and be able to export this data for accounting. It must be able to read the health of the business at a glance: how many leads are lost, how many enrolments are abandoned, which courses are viewed but rarely opened. And it must handle corporate clients, who do not buy one course at a time but send whole teams of employees for training.

2.3 The integration gap

The core difficulty is that these two sides are normally handled by separate tools-an e-commerce platform and a CRM-which must then be synchronised. Every synchronisation point is a place where data can drift: a purchase recorded in the shop but missing from the CRM, a discount applied on the invoice but not on the lead's

record, a support ticket that never reaches the sales team. Manual reconciliation is slow and error-prone.

2.4 Problem statement

The problem, therefore, is to build a single web application in which the customer-facing storefront and the business-facing CRM are not two integrated systems but *one* system, so that every customer action is captured exactly once and automatically drives the corresponding business processes-enrolment, invoicing, lead scoring, auditing, and support-without any manual reconciliation. The application must additionally remain approachable for non-technical staff, keep the two roles (customer and administrator) strictly separated, and stay fully functional even when optional services such as the AI assistant are switched off.

Chapter 3

Similar Approaches

Several categories of existing software address parts of the problem, but none of them covers it end to end in a single product. General-purpose course marketplaces such as *Udemy* and *Coursera* offer an excellent storefront, catalogue, reviews, and checkout, but they are aggregators: the individual training provider does not own the customer relationship and has no integrated CRM to score leads, manage corporate accounts, or issue its own invoices. Learning Management Systems such as *Moodle* focus on delivering and tracking course *content*-enrolments, grades, completion-rather than on selling courses and managing the commercial relationship; e-commerce and CRM features are, at best, plug-ins bolted on afterwards. Dedicated CRM platforms such as *Salesforce* or *HubSpot* are extremely strong on the business side-contacts, pipelines, lead scoring, reporting-but they are generic: they have no notion of a course catalogue, a trainer availability calendar, or a public storefront, and connecting them to a separate shop reintroduces exactly the integration gap described in the previous chapter. Finally, self-hosted e-commerce engines such as *WooCommerce* can sell anything, including courses, but treat a course as an ordinary product and know nothing about lead stages, tutoring sessions, or corporate training.

TrainingIT differs from all of these by unifying the storefront and the CRM in a *single* application built around one shared data model, so that a purchase, a review, a tutoring booking, or a support ticket is recorded once and immediately visible on both sides. It further specialises the generic CRM notions for the training domain-course sessions, trainer calendars, per-hour tutoring with employee discounts, and course-level click-through analytics-and adds an optional AI layer for recommendations and translation. Its domain core is deliberately organised around classic design patterns, which keeps the unified model maintainable as it grows.

Chapter 4

Proposed Solution

The proposed solution is a single web application, TrainingIT, whose two portals share one back end and one database. The system context is shown in Figure 4.1.

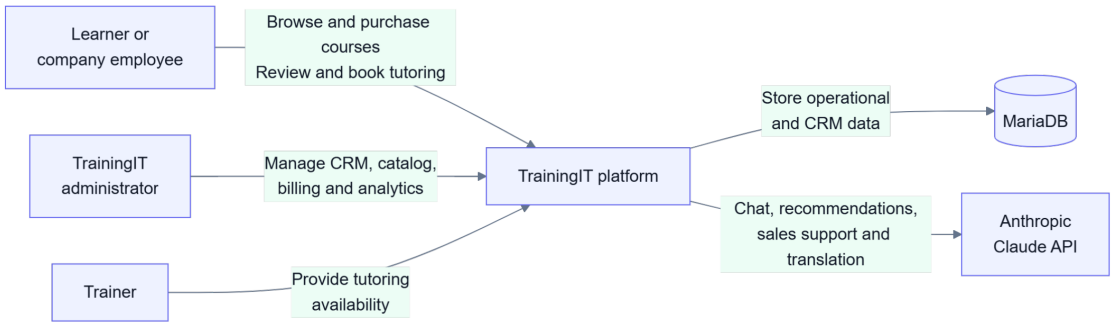


Figure 4.1: System context: the actors that interact with the TrainingIT platform and the external services it relies on.

4.1 One application, two experiences

The application decides which experience to show purely from the e-mail address used at login, and the two roles never mix. A **USER** (a learner or a company employee) lands on the public portal and can browse and buy courses, review them, book one-to-one tutoring, manage sessions, and report issues. An **ADMIN** (a member of the TrainingIT team) lands on the administration portal and works with seven management tabs: contacts, courses, purchases, invoices, analytics, employees, and issues.

If an administrator requests a client route, or a client requests an administrator route, the application redirects them back to their own portal.

4.2 Everything is captured once

Because both portals are served by the same back end, every customer action is recorded exactly once and, where relevant, automatically triggers the corresponding business processes. Registering creates a lead in the CRM; buying a course creates

an enrolment that appears instantly in the administrator's *Purchases* tab; booking a tutoring session generates an invoice by itself; reporting an issue drops a ticket straight into the administrator's *Issues* tab. This "one click, several reactions" behaviour is implemented with an event-driven core (Chapter 5) and is the central idea of the solution.

4.3 Optional intelligence

On top of this core, an optional AI layer powered by the Anthropic Claude model provides a conversational assistant, a visitor recommendation quiz, per-company training recommendations, and live translation of the interface into any language. This layer is strictly additive: if it is disabled, its buttons simply do not appear and every other feature continues to work.

Chapter 5

Application Architecture

TrainingIT is organised into three layers that communicate through REST APIs and a real-time event stream, plus an optional external AI service. The overall architecture is shown in Figure 5.1.

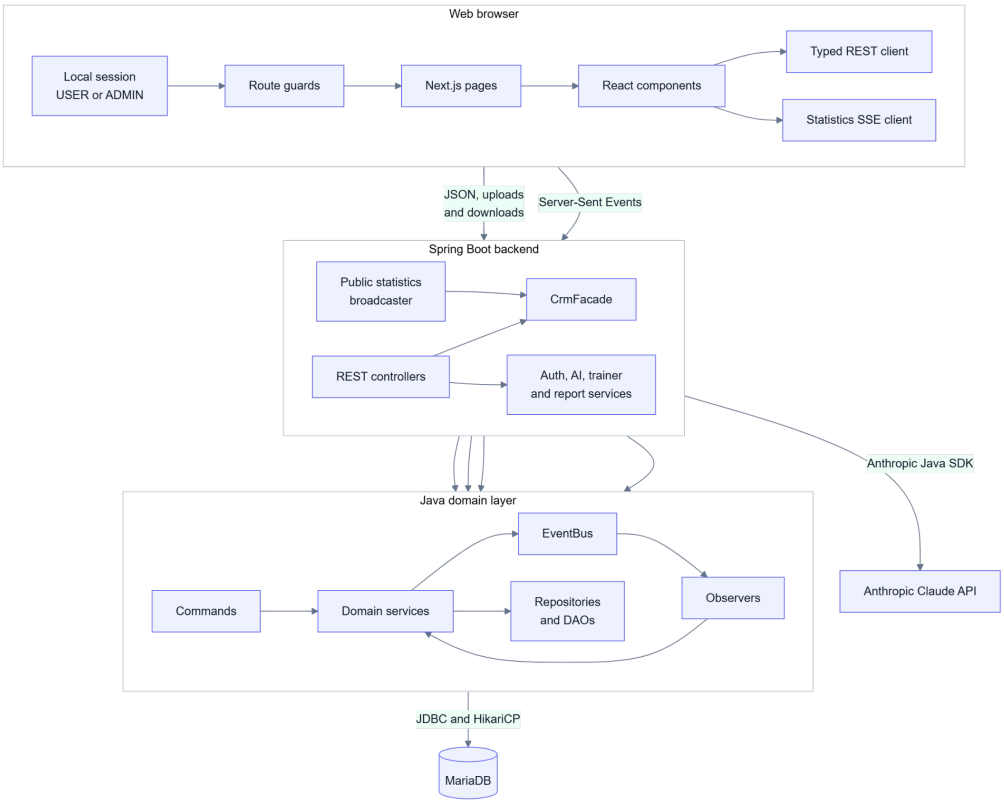


Figure 5.1: Application architecture: the Next.js client, the Spring Boot back end, the pattern-based Java domain layer, the MariaDB database, and the Claude API.

5.1 Presentation layer (front end)

The front end is a Next.js and React application that renders both the client and the administrator portals. It holds the browser session (USER or ADMIN), guards routes with an authentication guard, calls the back end through a typed REST

API client, and subscribes to a Server-Sent Events (SSE) stream for live public statistics. It also owns the cross-cutting features that work on every page: the light/dark theme switch and the live translation selector.

5.2 Application layer (back end)

The back end is a Spring Boot service that exposes REST controllers under `/api/...` The controllers are thin: they validate input and delegate to the domain layer, either through a single facade (`CrmFacade`) for core CRM operations or through specialised web services for authentication, AI, trainers, and reports. A dedicated broadcaster pushes recomputed public statistics over SSE while at least one browser is subscribed.

5.3 Domain layer

The domain layer is the heart of the system and is built as a demonstration of object-oriented design patterns. Write operations are wrapped as *commands* and dispatched by a command invoker; business logic lives in singleton *services*; and cross-entity reactions are decoupled through an *EventBus* that notifies a set of observers (audit logging, welcome and confirmation e-mails, invoice generation, and lead-score updates). Persistence is handled by repositories and DAOs. The domain patterns and the reactions they wire together are shown in Figure 5.2 and detailed in Chapter 8.

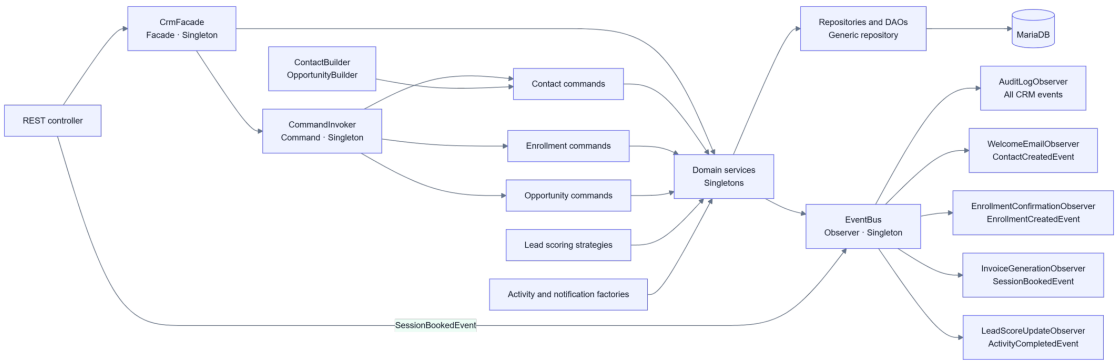


Figure 5.2: Domain patterns and event reactions: the Facade dispatches commands and services, which publish events that fan out to independent observers.

5.4 Persistence layer

All data is stored in a MariaDB relational database, accessed through the MariaDB JDBC driver with a HikariCP connection pool. The schema-covering both the CRM tables and the web-feature tables-is created and updated automatically at application start-up, and the data survives restarts.

5.5 Role-based navigation

Access control is driven by the account type resolved at login. As shown in Figure 5.3, an unauthenticated visitor can only reach the login, registration, and password-recovery screens; after signing in, the e-mail is matched against the administrators, contacts, and employees, a USER or ADMIN session is created, and the visitor is routed to the corresponding portal. An employee e-mail resolves to (or creates) a portal contact and applies the employee discount.

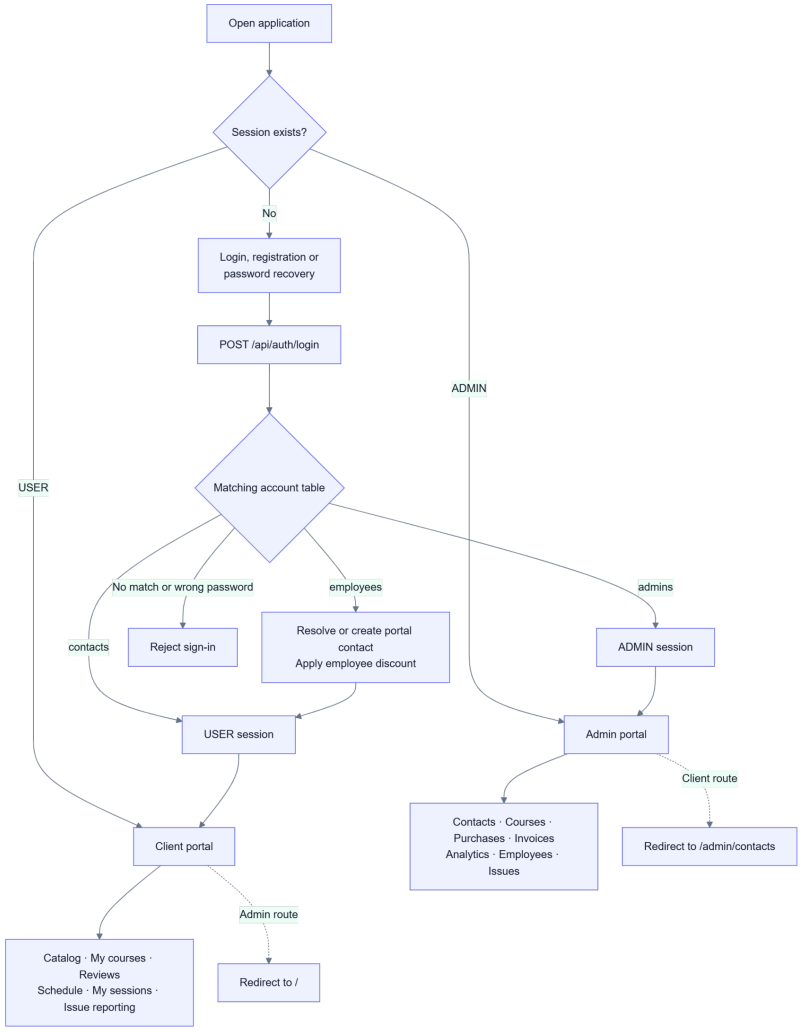


Figure 5.3: Role-based navigation: how the login result determines the session role and the destination portal.

Chapter 6

Application Features

This chapter walks through the features of the two portals as a user would meet them.

6.1 Account, authentication and roles

The whole application is protected: an unauthenticated visitor reaches only the authentication screens. There are three of them-*Log in*, *Register*, and *Forgot password*. Registration is a self-service form split into thematic cards (personal details, contact details, address, learning profile, password, and a mandatory GDPR consent); on success it creates an `INDIVIDUAL` contact in the CRM, computes its lead score, and logs the user in automatically. Login inspects the e-mail, verifies the password, and returns the role, which decides the destination portal.

6.2 The client portal

After signing in as a client, the header exposes the navigation menus, and two floating buttons hover in the corners.

6.2.1 Courses (home page)

The richest page of the portal. It opens with a personalised greeting and a band of four live figures (number of courses, number of learners, average rating, and one-to-one mentoring), which update themselves in real time over SSE without a page reload. Below, an optional AI quiz ("Not sure where to start?") asks for interests, level and goal, and returns recommended courses with a match score and a reason. The catalogue itself is a filterable grid of course cards; each card can expand to show a detailed description, exposes the average rating and review count, links to a dedicated reviews page, and offers a one-click *Enroll* button. The first time a card is expanded, a click is counted for the catalogue click-through analytics.

6.2.2 My courses

The learner's personal shelf: every purchased course, listed one under another, visible only to its owner. This is the single place in the application where reviews

are written-one review (a rating from 1 to 5 plus a comment) per purchased course. From here the learner can also start booking a tutoring session.

6.2.3 Online sessions and My sessions

The booking flow reserves a one-to-one tutoring session in four steps: choose a trainer, choose a day, choose the start time and duration, then confirm and pay. Trainers work Monday to Saturday between 08:00 and 20:00; past days, Sundays and fully booked days are greyed out. The price is $10 per hour, and employees of partner companies automatically receive a discount, shown as a struck-through price next to the reduced one. Confirming the transfer books the session, generates the invoice, and marks the slot as taken. *My sessions* lists the learner's booked sessions and allows cancellation, which frees the slot again.

6.2.4 Floating buttons: assistant and issue reporting

A pink round button in the bottom-right corner opens an AI chat assistant powered by Claude; a flag in the bottom-left corner opens a "Report an issue" panel. Each reported issue lands instantly in the administrator's *Issues* tab, tagged with the reporter's name and e-mail.

6.3 The administrator portal

The administration portal is reachable only with an ADMIN session and is organised as seven tabs. **Contacts** - the heart of the CRM: all individuals and companies, each with a lead status (`NEW`, `CONTACTED`, `INTERESTED`, `QUALIFIED`, `ENROLLED`, `LOST`) and an automatically computed lead score; searchable and exportable to CSV, Excel, or PDF. **Courses** - the internal, tabular view of the same catalogue the clients see. **Purchases** - the full history of enrolments, with the learner, the course, the date and the rating left, plus a total-orders card. **Invoices** - invoices generated automatically for one-to-one sessions, with the employee discount applied where relevant, each downloadable as a PDF. **Analytics** - colour-coded health metrics (lead churn, enrolment drop-out, opportunity loss, catalogue click-through rate) with a written interpretation, a per-course click-through table, and demographic bar charts. **Employees** - management of partner-company employees, including bulk import from an Excel file (with a per-row error report) and AI course recommendations for the whole team. **Issues** - the tickets sent by clients, which the team can mark solved, reopen, or delete (deletions require an explicit confirmation because they cannot be undone).

6.4 Cross-cutting features

Two capabilities work on every page regardless of the menu. The **live translation** selector translates the whole page-including dynamically loaded content-into any language using Claude, while preserving brand names, people, e-mails and numbers. The **light/dark theme** switch flips the theme instantly and remembers the choice locally, applying it before the first paint so there is no flash of the wrong theme.

6.5 Student Contributions

I then went further by integrating Claude from Anthropic, giving the CRM three practical, fully implemented AI features: course recommendations for partner-company employees, course recommendations for individual learners, and a chatbot. The employee feature analyses roles and interests to propose suitable team training, while the individual feature matches each learner with courses suited to their interests, experience level and goals. The chatbot uses the current catalogue as context to answer questions about courses, enrolment and corporate training. I implemented the backend prompts, REST endpoints, structured JSON response handling and corresponding user interfaces, while ensuring that the rest of the application remains operational when AI is not configured.

Chapter 7

Technologies Used

The application is a full-stack web system. This chapter lists the technologies used in each layer and the reason for each choice.

7.1 Front end

Next.js 16 and **React 19** - the component framework for both portals, with client components that call the REST API. **TypeScript 5** - static typing across the client code base. **Tailwind CSS v4** - utility-first styling for the custom visual theme (fuchsia pink and black on a warm off-white background) and the dark mode.

7.2 Back end

Java 17 - the implementation language of the domain and web layers. **Spring Boot 3.5** - the application framework that exposes the REST controllers and the SSE stream and wires the components together. **Spring Validation** - declarative validation of incoming request payloads. **Lombok** - reduction of boilerplate in the entity and DTO classes. **SLF4J with Logback** - application logging.

7.3 Persistence

MariaDB - the relational database that stores all operational and CRM data. **MariaDB JDBC driver** with **HikariCP** - direct, pooled database access. The domain layer uses hand-written repositories and DAOs rather than an object-relational mapper, keeping the SQL explicit and the pattern-based design visible.

7.4 Reporting and artificial intelligence

Apache POI - generation and parsing of Excel spreadsheets (contact/report exports and the employee bulk import). **OpenPDF** - generation of PDF invoices and PDF exports. **Anthropic Claude (anthropic-java SDK)** - the optional AI layer (assistant, recommendations, translation), using the `claude-opus-4-8` model. The client is initialised only when an API key is present; otherwise the AI features are cleanly disabled.

Chapter 8

Implementation Details

This chapter describes how the design of Chapter 5 is realised in code. The server-side code base contains roughly 11,500 lines of Java organised by responsibility (builders, commands, DAOs, factories, models, observers, patterns, repositories, services, strategies, validation, and the web layer), and the client roughly fifty TypeScript/React modules.

8.1 Design patterns in the domain layer

The domain core is a deliberate demonstration of object-oriented design patterns. Table 8.1 summarises where each one appears.

8.2 The event-driven core

The “one click, several reactions” behaviour is implemented with the Observer pattern. When a service completes a meaningful operation it publishes an event on the `EventBus`; the bus notifies every registered observer, and each observer reacts independently. A representative example is the invoice observer, which listens for a booked session and generates the invoice without ever blocking the booking itself:

```
1  public class InvoiceGenerationObserver implements Observer<
       CrmEvent> {
2      @Override
3      public void update(CrmEvent event) {
4          if (!(event instanceof SessionBookedEvent)) return;
5          MeditationSession session = ((SessionBookedEvent)
               event).getSession();
6          if (session == null || session.getId() == null)
               return;
7          try {
8              InvoiceService.getInstance().generateForSession
                   (session);
9          } catch (Exception ex) {
10             logger.error("Error auto-generating the invoice
                   for session {}",
11                     session.getId(), ex);
12         }
```

Table 8.1: Design patterns used in the domain layer.

Pattern	Role in the application
Observer	An `EventBus` broadcasts CRM events to independent listeners (audit log, welcome e-mail, enrolment confirmation, invoice generation, lead-score update).
Command	Each write operation (`CreateContact`, `EnrollContact`, opportunity commands, ...) is encapsulated as an executable command and dispatched by a command invoker.
Strategy	Lead scoring is computed differently for individuals (`SimpleLeadScoringStrategy`) and companies (`B2BLeadScoringStrategy`), selected through a `LeadScoringContext`.
Factory	Notifications and activities are produced by factories, avoiding direct coupling to concrete types.
Builder	Complex objects (`Contact`, `Opportunity`) are assembled step by step by dedicated builders.
Facade	`CrmFacade` offers a single, simple entry point over all CRM services.
Chain of Responsibility	Contact validation is a chain of validators (e-mail, phone, GDPR, contact type).
Generic DAO / Repository	A generic persistence abstraction is reused across entities.
Singleton	Services, the command invoker and the event bus are single shared instances.

```
13      }
14  }
```

Listing 8.1: The invoice-generation observer reacts to a booked-session event; a failure is logged but never rolls back the booking.

Invoice creation is *idempotent*-if an invoice already exists for the session it is not created again-and the observer's failure is isolated, so a problem while invoicing never prevents the tutoring session from being booked. The full set of events includes contact creation, enrolment creation, session booking, activity completion, and lead-status and opportunity-stage changes.

8.3 Data model

The data model, shown in Figure 8.1, spans both the CRM tables and the web-feature tables. A `Contact` is either an `INDIVIDUAL` or a `CORPORATE` company; corporate contacts employ `Employee` records. A `Course` offers `CourseSessions`, into which contacts are placed through an `Enrollment`. Opportunities and activities

model the sales pipeline; trainers deliver tutoring sessions, which in turn generate session invoices.

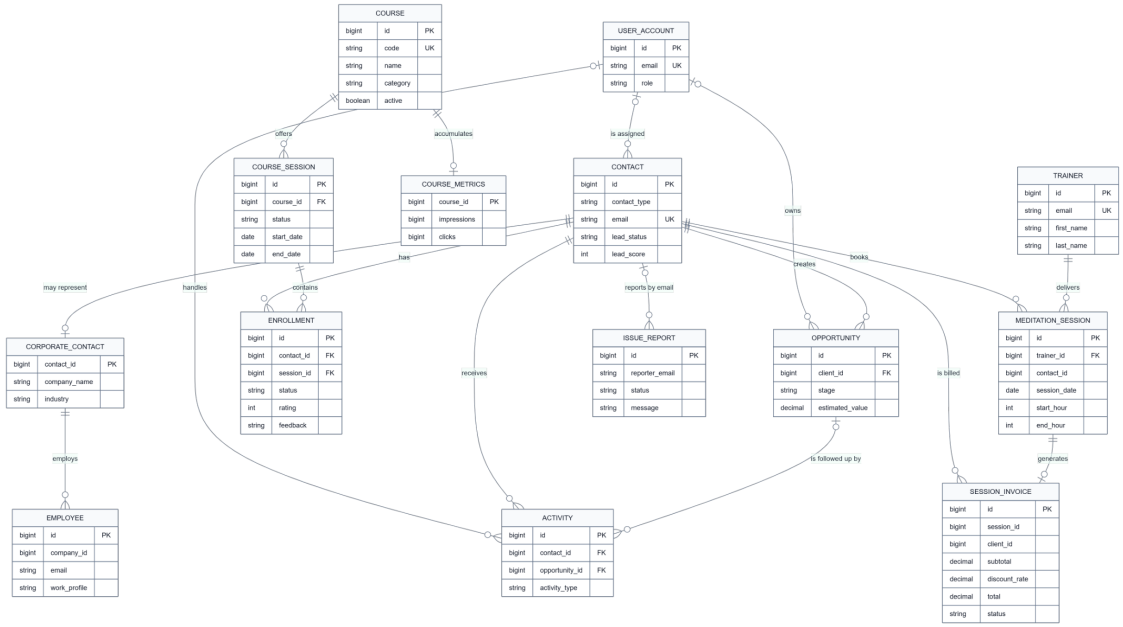


Figure 8.1: Core data model: the CRM and web-feature entities and their relationships.

A design decision worth highlighting is that **reviews do not have their own table**: they live directly on the enrolment as a `rating` column and a `feedback` column. This enforces a single review per (contact, course) pair naturally-resubmitting overwrites the previous one-and a rating between 1 and 5 counts as a valid review, while the value 0 means "not yet reviewed".

8.4 AI integration

The AI layer is a thin wrapper (`ClaudeClient`) around the official Anthropic Java SDK. The key is read from configuration; when it is absent, the client is left `null`, `isEnabled()` returns `false`, and every AI endpoint reports that the feature is off instead of crashing. The same client serves four capabilities-the chat assistant, the visitor recommendation quiz, per-company recommendations, and live translation-so a single key enables all of them at once.

8.5 Real-time statistics and reporting

Public statistics (course count, learner count, average rating) are pushed to the browser through Server-Sent Events: a broadcaster recomputes the figures while at least one client is subscribed and streams only the changed values, so the home page updates without polling. On the reporting side, contact lists are exported with Apache POI (Excel) and OpenPDF (PDF), and the same libraries back the PDF invoices and the employee Excel import.

Chapter 9

Testing

Because the correctness of TrainingIT depends less on isolated algorithms and more on the correct propagation of a user action through the whole system, the testing effort concentrated on input validation, robustness of the event-driven core, and functional verification of the end-to-end flows.

9.1 Input validation

Input is validated on two levels. On the client, forms check for missing fields, password confirmation and the mandatory GDPR consent before anything is sent. On the server, incoming payloads are validated with Spring Validation, and contact data additionally passes through a Chain-of-Responsibility validator that checks the e-mail, the phone number, the GDPR flag, and the contact type in sequence, so that an invalid contact never reaches the database.

9.2 Robustness by design

Several behaviours were verified specifically because they protect the system under failure. Invoice generation is *idempotent*, so a repeated event cannot produce a duplicate invoice. Observer failures are *isolated*: an error while generating an invoice is logged but does not roll back the booking that triggered it. The AI layer *degrades gracefully*: with no API key the AI buttons disappear and the rest of the application is unaffected. Deletions in the *Issues* tab are guarded by an explicit "this cannot be undone" confirmation. Each of these was checked by exercising the corresponding failure condition.

9.3 Functional testing

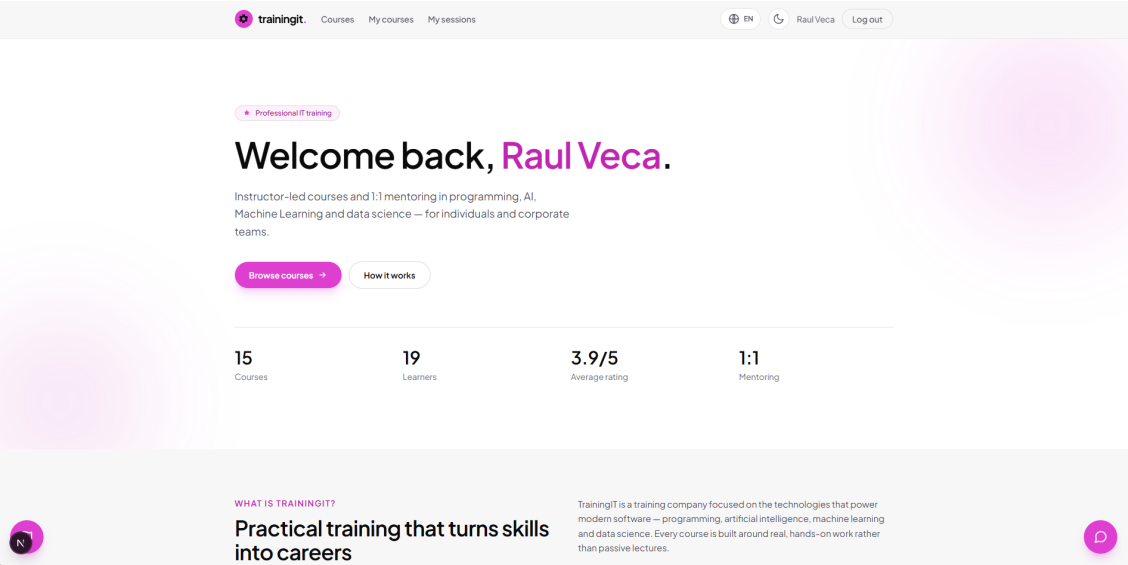
The main user journeys were tested manually against a running instance backed by a real MariaDB database: registration creating a lead; login routing a USER and an ADMIN to the correct portal; a one-click enrolment appearing immediately in the *Purchases* tab; a review written from *My courses* updating the course's average rating live; a tutoring booking producing an invoice with the correct employee discount; an issue reported from the client reaching the *Issues* tab; and the export of contacts to CSV, Excel and PDF. The project also includes the Spring Boot test

harness (`spring-boot-starter-test`) as a foundation on which an automated test suite can be built; extending this into full unit and integration coverage is identified as future work in Chapter 11.

Representative functional test evidence

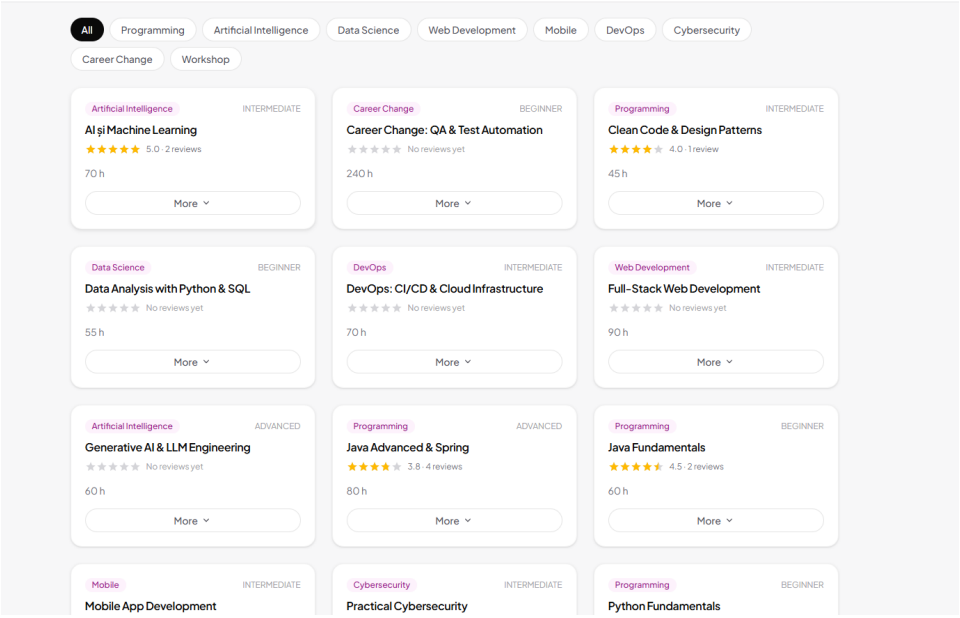
The screenshots below record the visible results returned by the learner and administrator interfaces during manual end-to-end verification. Each image is described together with the functionality that was exercised successfully.

(a) Authenticated learner dashboard



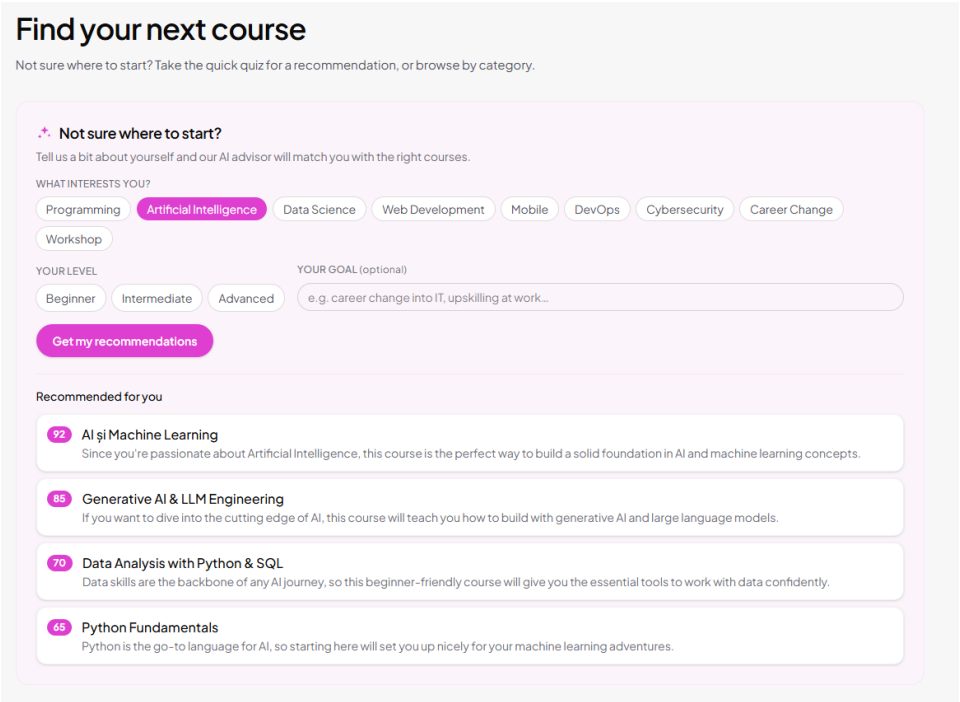
The screenshot shows the authenticated learner home page with the personalised ‘Welcome back, Raul Veca’ greeting, learner navigation, live course and learner statistics, the average rating, and the mentoring indicator. During manual testing, a USER account was routed to this page after login and the navigation and dashboard data loaded correctly, confirming that authenticated learner access works.

(b) Course catalogue and category filters



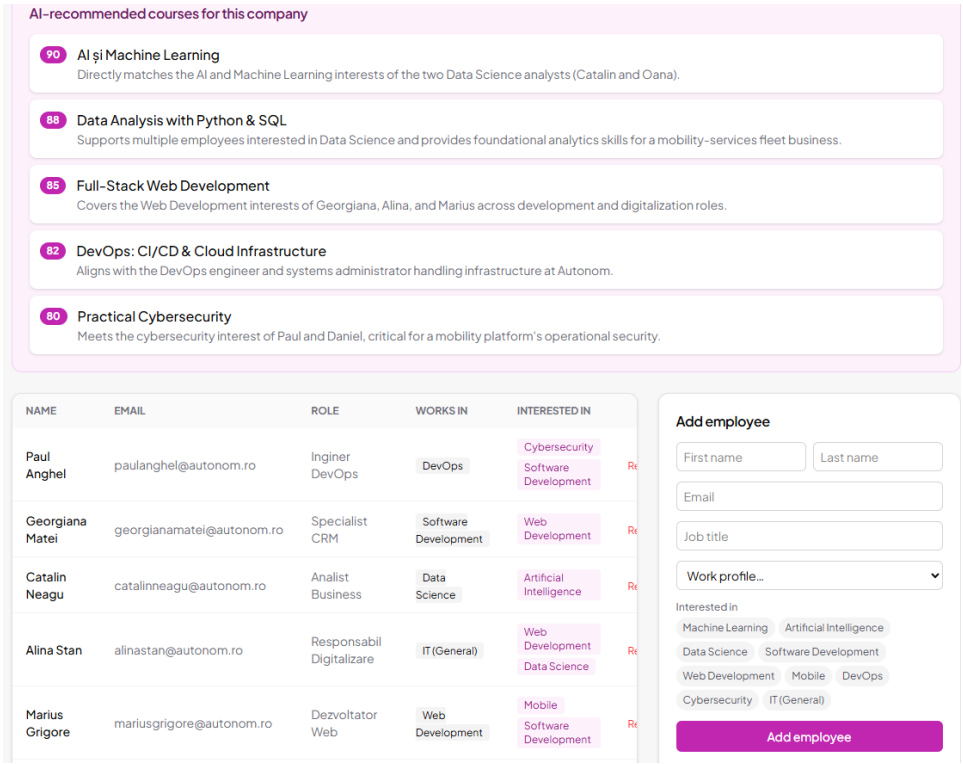
The image displays the course catalogue with category filters and a grid of course cards showing category, difficulty level, rating, duration, and the More control. Manual testing confirmed that the catalogue loads the stored courses and that selecting a category updates the visible results, so course discovery and filtering work.

(c) Personalised AI course recommendations



The screenshot shows the Find your next course adviser with Artificial Intelligence selected and a ranked response containing match scores and explanations for four courses. Manual testing confirmed that submitting learner preferences calls the recommendation endpoint and renders the ranked matches, verifying that the personalised AI recommendation flow works.

(d) Company-level AI recommendations



The administrator view shows five scored, AI-recommended courses above the employee table, whose rows provide roles, work areas, and training interests, together with the Add employee form. Manual testing confirmed that these employee profiles are analysed into ranked company training suggestions and that the resulting list is displayed correctly, so the company-level recommendation functionality works.

Chapter 10

Usage Examples

This chapter follows three representative scenarios, inspired by the sample data shipped with the application, that show the system in action.

10.1 A career changer, guided by the assistant

Alexandru works in sales and wants to move into IT but does not know where to start. He opens the chat assistant and describes his goal; the assistant recommends two beginner-friendly courses. He registers-appearing in the CRM as a new lead with a computed score-buys the more accessible course, which shows up at once in the *Purchases* tab, and books a one-to-one session for personal guidance, which automatically produces an invoice. In a few minutes an undecided visitor has become a paying customer, and the company has both a new lead record and an invoice, with no manual data entry.

10.2 An advancing developer

Cristina is a junior developer who arrived through a friend's recommendation, and therefore carries a high lead score. She knows what she wants and asks the assistant directly for advanced courses; she also learns she can book a one-to-one session. She buys quickly. From the company's point of view the cost of acquiring her was almost nothing, which illustrates why referrals from satisfied learners are the cheapest form of marketing-something the analytics lead-source chart makes visible.

10.3 A corporate client training a team

BankTech Solutions is an IT company that wants to train its development team. An administrator imports dozens of employees from a single Excel file in the *Employees* tab, then asks the AI for course recommendations for the whole team. A B2B opportunity is created in the CRM, and the employees purchase with their automatic 60% discount, each one-to-one session producing an invoice by itself. A single company thus brings in many learners at once, while the CRM keeps track of the whole team, computes the discounts, and issues the invoices automatically.

10.4 The purchase and review flow

Underneath these scenarios, the purchase and review sequence is the same, shown in Figure 10.1. Confirming a purchase resolves (or creates) the contact, finds or creates the course session, saves the enrolment, and publishes an enrolment event; the statistics broadcaster then pushes the changed figures to every subscribed browser. Submitting a review validates the prior purchase and saves the rating and comment onto the enrolment.

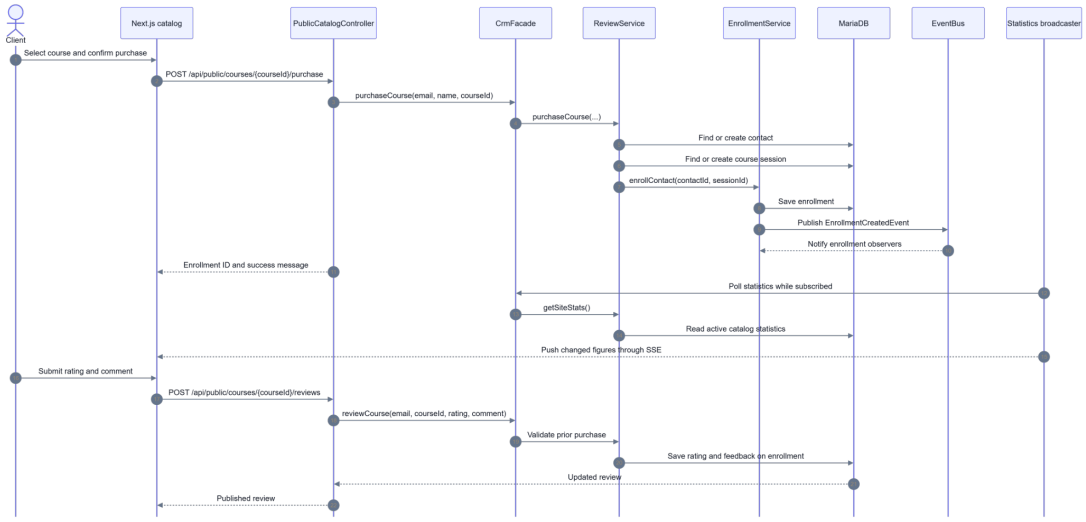


Figure 10.1: Course purchase and review flow, from the client's confirmation to the live statistics update and the published review.

Chapter 11

Conclusions and Future Development Directions

11.1 Conclusions

This thesis presented TrainingIT, a web-based application for selling programming courses that unifies, in a single product, an online course marketplace and a complete CRM back office. The central result is that the two sides share one back end and one data model, so that every customer action-registration, purchase, review, tutoring booking, or support ticket-is captured exactly once and automatically drives the corresponding business processes, eliminating the integration gap that separate shop-and-CRM setups suffer from. A visitor becomes a tracked lead on registration and a customer on the first purchase, and each subsequent action propagates cleanly into the administration portal.

The application meets the objectives set out in the introduction. It delivers a polished client portal and a seven-tab administrator portal; its Java domain layer is organised around classic design patterns and an event-driven core that makes "one click, several reactions" both natural and robust; it persists all data in MariaDB; and it integrates an optional AI assistant and live translation that degrade gracefully when switched off. What makes the system distinctive is the combination of an approachable interface, a seriously engineered CRM core, and an optional layer of artificial intelligence, all in one coherent product.

11.2 Future development directions

Several directions would extend the work.

Automated test suite. Building unit and integration tests on top of the existing Spring Boot test harness, to cover the commands, observers, scoring strategies and REST endpoints, and to enable continuous integration.

Stronger authentication. Adding token-based sessions, password hashing at rest, e-mail verification, and a genuine password-reset flow with expiring links.

Richer analytics. Historical trends, cohort analysis, and exportable dashboards beyond the current point-in-time metrics.

Deeper AI. Grounding the assistant in the live catalogue, adding conversation memory, and using AI to summarise support issues for the administrators.

Deployment and scaling. Containerising the two services, externalising configuration, and preparing the application for a cloud deployment.

Bibliography

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[2] Spring Boot Reference Documentation. `https://docs.spring.io/spring-boot/` Accessed: April 9, 2026.

[3] Next.js Documentation. `https://nextjs.org/docs` Accessed: May 28, 2026.

[4] React Documentation. `https://react.dev/` Accessed: May 16, 2026.

[5] MariaDB Server Documentation. `https://mariadb.com/kb/en/documentation/` Accessed: April 24, 2026.

[6] Tailwind CSS Documentation. `https://tailwindcss.com/docs` Accessed: July 6, 2026.

[7] Anthropic Claude API Documentation. `https://docs.anthropic.com/` Accessed: June 17, 2026.

List of Figures

4.1 System context: the actors that interact with the TrainingIT platform and the external services it relies on. 7

5.1 Application architecture: the Next.js client, the Spring Boot back end, the pattern-based Java domain layer, the MariaDB database, and the Claude API. 9
5.2 Domain patterns and event reactions: the Facade dispatches commands and services, which publish events that fan out to independent observers. 10
5.3 Role-based navigation: how the login result determines the session role and the destination portal. 11

8.1 Core data model: the CRM and web-feature entities and their relationships. 21

10.1 Course purchase and review flow, from the client's confirmation to the live statistics update and the published review. 28

Appendix A

Appendix - Generative AI Tools Usage

An overview of how generative AI tools were used in this thesis is presented in Table A.1.

AI Tool	Use case	Used in	Remarks
Claude (Anthropic)	Suggestions for code cleanup, local refactoring, and removal of redundant code	Targeted code areas of the existing application	Details in section A.1; all changes were manually reviewed

Table A.1: Overview of generative AI usage

A.1 Claude Details

The purpose of using Claude was limited to suggestions for code cleanup, local refactoring, and the removal of redundant code. It was not used to generate the thesis text, the system architecture, or new application functionality.

The method consisted of presenting already-written, targeted code areas from the existing application and asking for focused improvement suggestions on those specific fragments.

Verification was performed locally by running the affected areas of the application and, where appropriate, by executing relevant tests. Recommendations were accepted only when they were compatible with the existing architecture.

A.2 Claude Usage Screenshots

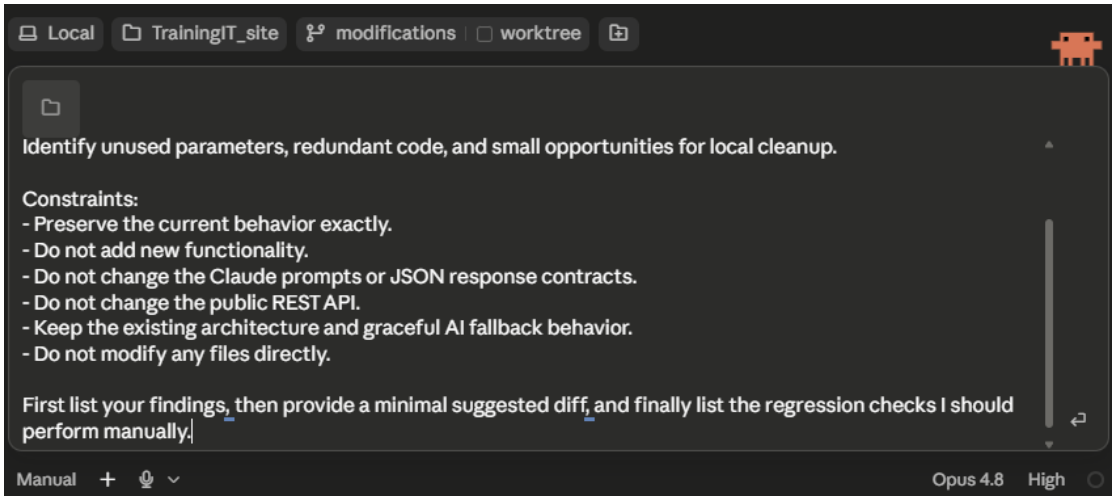


Figure A.1: Targeted code-cleanup prompt and constraints

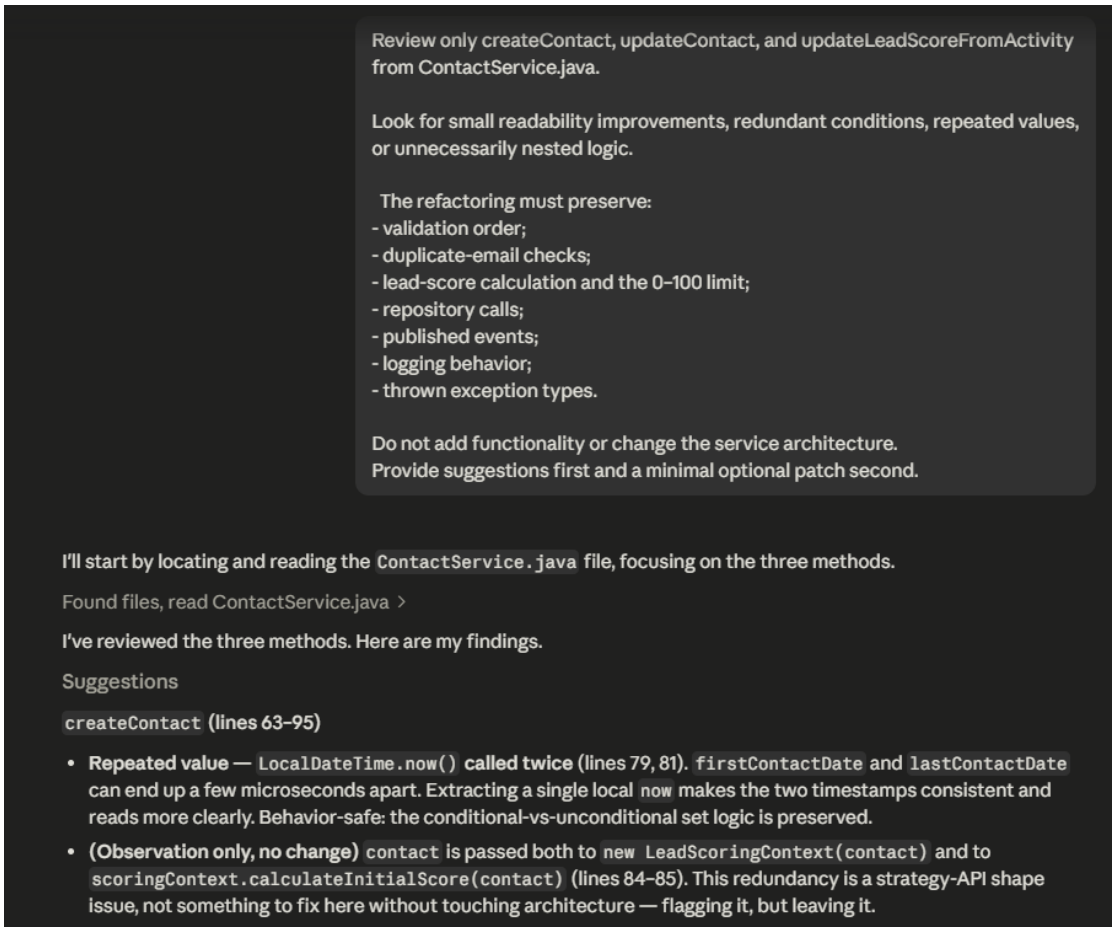


Figure A.2: Focused review request and Claude findings

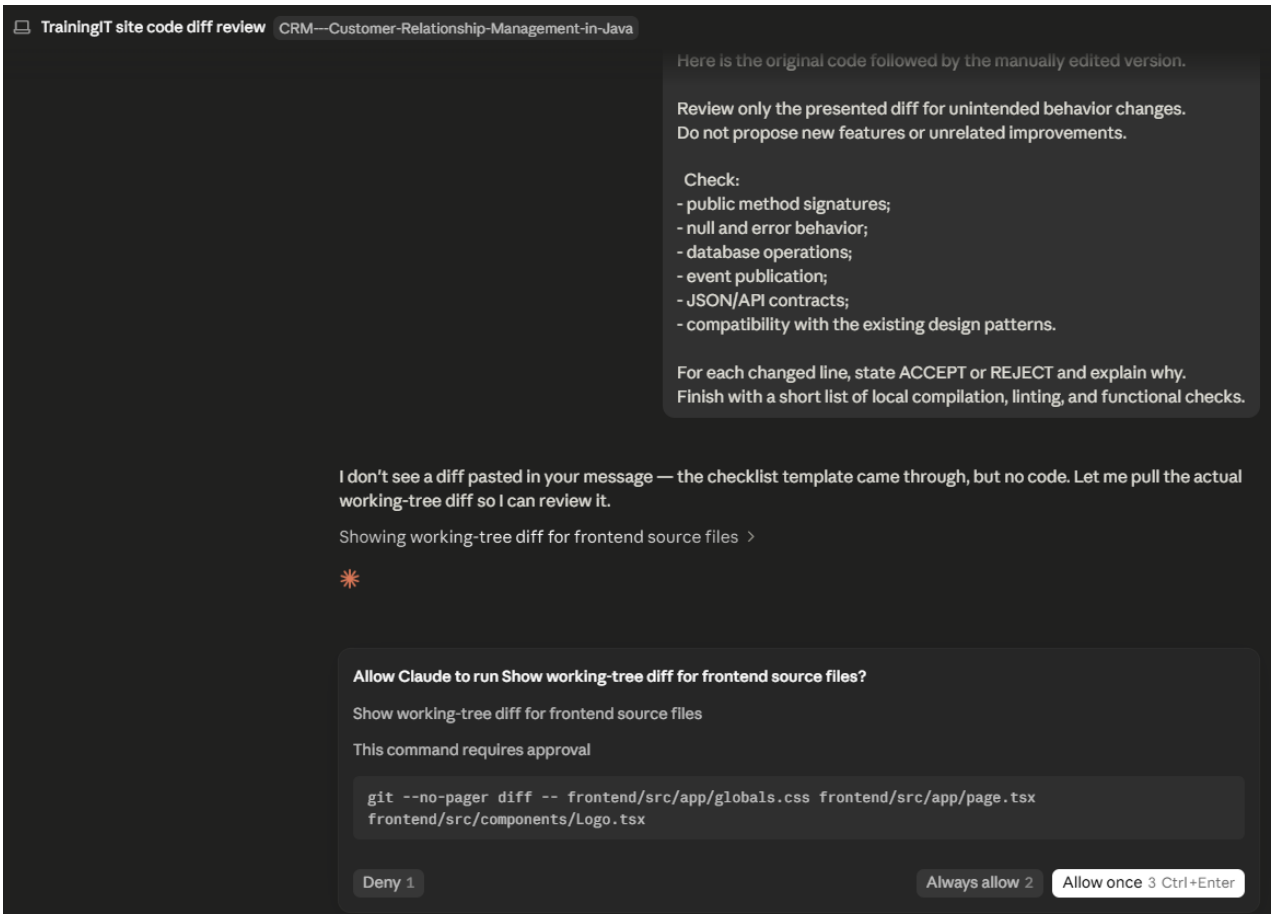


Figure A.3: Diff review request and command-approval step